

scripts/check-cycle-coverage.sh

— User Guide

Last verified: 2026-07-01 (re-verified against the committed §6.AK implementation at HEAD — flags/exit-codes/file-formats below match scripts/check-cycle-coverage.sh; prior: 2026-06-26 gate landing)

Inheritance: HelixConstitution §11.4.18 (script documentation mandate) · enforces Lava CLAUDE.md §6.AK clause 1 (Cycle-Coverage Device Gate)

Overview

check-cycle-coverage.sh is the mechanical implementation of the CM-CYCLE-COVERAGE-INTERSECTION gate (§6.AK / §6.AK-debt). It refuses a distribute unless **every CHANGELOG-claimed user-visible fix for the current version has an EXECUTED + PASSED covering device Challenge** in the matching §6.Z evidence file for the **same commit SHA**, recorded within the last 24 hours.

It exists to catch the class of bluff that shipped Lava-Android-1.3.12-1076 (commit 627a0d58): the §6.Z device gate executed ONLY Challenge00 (cold-start) while the CHANGELOG claimed fixes to search, provider-selection, and onboarding — a green gate that proved nothing about the shipped value.

The gate intersects two sets and requires $\text{claim-set} \subseteq \text{executed-pass set}$:

- **CLAIM-SET** — the covering Challenge declared for each CHANGELOG-claimed user-visible fix, read from a per-cycle cycle-coverage-map YAML.
- **EXECUTED-PASS SET** — Challenges that ran and PASSED on a gate-eligible device in the §6.Z evidence file (verdict PASS and runner != host-direct).

Any claim with no covering executed+passed Challenge → reject. It parses YAML and evidence line-by-line with grep/sed/bash (no yq dependency).

Usage

```
# Auto-detect everything (version from app/build.gradle.kts,
# evidence dir from channel,
# map next to the evidence file, HEAD from git, "now" from the
# system clock):
./scripts/check-cycle-coverage.sh --strict

# Explicit version + evidence dir (the form firebase-distribute.sh
# Gate 7 uses):
./scripts/check-cycle-coverage.sh --version="1.3.13-1077" \
  --evidence-dir=".lava-ci-evidence/distribute-changelog/firebase-
  app-distribution" --strict

# Fully pinned inputs (the form the hermetic tests use – no git,
# no clock, no gradle):
./scripts/check-cycle-coverage.sh --version="9.9.9-9999" \
  --evidence-dir "<dir>" --map "<map.yaml>" \
  --head "<sha40>" --now-epoch "<epoch>"
```

Flags

Flag	Default	Meaning
--version <name-code>	auto-detect from app/ build.gradle.kts (versionName- versionCode)	Version under test, e.g. 1.3.12-1077.
-- evidence- dir <path>	resolved from channel (see below)	Directory holding <version>- test-evidence.{md,json}.
--map <path>	auto-resolve (see below)	Path to the cycle-coverage-map YAML.
--head <sha>	git rev-parse HEAD	Commit under test. The evidence MUST declare a matching SHA; an absent or unknown SHA is a REFUSAL (exit 2), not a free pass. Overridable for hermetic tests.
	date +%s	

Flag	Default	Meaning
<code>--now-epoch <epoch></code>		"Now" (Unix epoch) for the $\leq 24h$ freshness check. Overridable for determinism.
<code>--strict</code>	on by default (STRICT=1)	Reject on any uncovered claim. The flag is accepted but the behavior is the default.
<code>-h, --help</code>	—	Print usage to stderr and exit 0.

Environment overrides (read once at startup, lower precedence than the flags): `LAVA_CYCLE_COVERAGE_HEAD` (initial value for `--head`) and `LAVA_CYCLE_COVERAGE_NOW_EPOCH` (initial value for `--now-epoch`). An unknown argument prints usage and exits 3.

Path resolution

- **Evidence dir** (when `--evidence-dir` is omitted): channel release $\rightarrow$ `.lava-ci-evidence/distribute-changelog/firebase-app-distribution`; any other channel $\rightarrow$ `.lava-ci-evidence/distribute-changelog/firebase-app-distribution-dev`.
- **Evidence file**: the first of `<EVIDENCE_DIR>/<version>-test-evidence.md` then `...-test-evidence.json` that exists (`.md` preferred). Missing $\rightarrow$ exit 2.
- **Map** (when `--map` is omitted): the first of `<EVIDENCE_DIR>/<version>-cycle-coverage-map.yaml` then `.lava-ci-evidence/<version>/cycle-coverage-map.yaml`. Missing $\rightarrow$ exit 2.

Exit codes

Code	Meaning
0	All claims covered by executed + PASSED, same-SHA, $\leq 24h$ device rows.
1	One or more claims lack a covering executed+PASSED Challenge (missing / FAIL / SKIP / host-direct), OR the evidence timestamp is older than 24h.
2	REFUSAL . Evidence file or map missing; map version: mismatch; the evidence declares no usable commit SHA (or only unknown); the declared SHA $\neq$ <code>--head</code> ; the map declares zero claims; the evidence format yields no parseable verdict

Code	Meaning
	record at all ; or an unknown CLI argument (including the removed <code>--channel</code>).

There is no exit 3. Every "the gate cannot determine an answer" condition is an exit-2 REFUSAL, because a gate that cannot determine a verdict has not established a PASS.

Input file formats

1. cycle-coverage-map (<vname>-<code>-cycle-coverage-map.yaml)

A small YAML block. The gate reads it line-by-line and only consumes two keys:

```
version: "1.3.12-1077"
claims:
  - bullet: "Search now returns real results"
    covering_challenge: "Challenge58SearchReturnsResults"
  - bullet: "Search filters follow onboarded providers"
    covering_challenge: "Challenge59SearchUsesOnboardedProviders"
```

- Each `covering_challenge`: value is a **required claim**.
- The preceding `bullet`: (if any) is used only for human-readable FAIL messages.
- A claim whose `covering_challenge` is empty/absent is itself an uncovered claim (→ exit 1) — that is the "CHANGELOG claims a fix but no Challenge was written" case the 1076 incident is about.
- A map with **zero** claims is a vacuous pass and REFUSES (→ exit 2). A cycle with genuinely no user-visible change must say so explicitly as a claim with its re-verification Challenge, per §6.AK clause 6.
- Surrounding single/double quotes on values are stripped before matching.

(Other keys such as `covers_issues`: may appear per the spec but are NOT read by the gate; only `covering_challenge`: values are matched.)

2. §6.Z evidence file (<vname>-<code>-test-evidence. {md,json})

The gate PARSES VERDICTS. It does not match names. (LVA-149 — see the history note at the top of the script.) It scans the evidence for records that STATE a result, in any of six shapes, and refuses if it finds none:

#	Shape	Where it is used in practice
R1	challenge: fqN=<FQN> verdict=PASS runner=containers-submodule	autonomous-QA / structured evidence
R2	"<TestName>": "PASS" (also FAIL / SKIP / ERROR, with optional trailing prose)	JSON covering_challenges maps
R3	"test_class": "<FQN>" ... "test_passed": true	JSON per-AVD attestation rows
R4	<TestName> ... **PASS** ...	markdown device-gate tables (the production format)
R5	<TestName>: PASS	Go / colon-verdict blocks
R6	<TestName>: tests="7" failures="0" errors="0"	JUnit summary lines (PASS iff tests>0, failures=0, errors=0)

A not run / – cell parses as SKIP, which is **not** a pass.

Coverage rule. A claim counts as covered only when a record whose name matches the map's covering name states PASS **and** the runner is not host-direct (host-direct is never gate-eligible per §6.AH/§6.AG).

Name matching is IDENTIFIER equality, not substring. Both sides are normalised to the last dot-segment with a single trailing Test stripped, so a map naming Challenge58SearchReturnsResults matches evidence naming

lava.app.challenges.Challenge58SearchReturnsResultsTest. Substring matching is deliberately gone: it is how the pre-LVA-149 gate let a bare NAME anywhere in the file stand in for a verdict.

A name mentioned in prose is not a verdict. If the evidence names a Challenge but states no result for it, that claim is uncovered (exit 1), and the message says so explicitly.

An unrecognised format REFUSES (exit 2) and names the file. If the parser extracts zero verdict records, it prints the offending path and lists the six shapes above so the operator can fix their evidence rather than guess. It never falls back to assuming a pass.

3. Commit-SHA binding (F2)

The evidence must declare the commit it was produced against. Accepted fields, in both markdown and JSON:

```
cycle-coverage: ... commit=<sha>
"commit_sha": "<sha>"          "tested_code_sha": "<sha>"
"artifact_code_sha": "<sha>"
commit_sha: <sha>              **Commit SHA:** <sha>
**Tested code SHA:** <sha>
```

Values are matched against `--head` by prefix in either direction (short SHAs are fine). If **any** declared SHA matches, the binding holds — this is recorded as bound via `literal:<sha>` in the PASS line. If SHAs are declared but none matches literally, a second pass tries the **ancestor-code-identity fallback** (§3.1 below) before refusing. If **none** is declared, or the only value is the literal `unknown` → exit 2 immediately (no fallback attempted — "unknown" is a refusal, not a free pass).

3.1 Ancestor-code-identity fallback (resolved 2026-09-23, closes §5's "open design question")

Tried only when the literal-match check finds no binding. For each declared SHA `S`: if `S` is a real, resolvable git object AND a genuine ancestor of `--head` (`git merge-base --is-ancestor`), AND every path that differs between `S` and `--head` (`git diff --name-only`) matches `^(\\.lava-ci-evidence/|docs/|CHANGELOG\\.md$)`, the binding holds — recorded as bound via `ancestor-code-identity:<sha>`. Any SHA that is not a resolvable ancestor, or whose diff touches so much as one file outside that allowlist, is rejected and the gate falls through to the exit-2 refusal unchanged.

This is deliberately **not** "any ancestor is close enough" — a declared SHA whose tree differs from `--head` in `app/build.gradle.kts`, any proguard file, or any other production path still refuses, exactly as before. The allowlist is what makes this code-identity, not leniency: the check answers "is the code genuinely the same," not "is this evidence old enough that we don't care."

Set `LAVA_CYCLE_COVERAGE_REPO_ROOT` to point the fallback's git plumbing at a different repository (used by the hermetic test suite's throwaway fixture repo, per §5 below); production invocations never set this and resolve `REPO_ROOT` exactly as before.

4. Freshness

A timestamp is read from `cycle-coverage: ... timestamp=, "timestamp", "authored_utc", or markdown Evidence authored: <date>`. If one is found and it is older than 24h relative to `--now-epoch`, the gate exits 1.

5. RESOLVED (2026-09-23) — pre-push Check 10 and the self-reference problem

This was an open design question from LVA-149 (2026-08-26) until 2026-09-23, recorded then as: a commit cannot embed its own SHA, so evidence authored in one commit and pushed via a later pointer-advancing commit could never bind via literal match, forcing `git push --no-verify` on every such cycle (see `.lava-ci-evidence/sixth-law-incidents/2026-09-22-cycle-coverage-self-reference-push-bypass.json` and the design doc at `docs/superpowers/specs/2026-09-22-cycle-coverage-self-reference-design.md` for the full history).

Resolved via a hybrid of the design doc's options 1 and 2: §3.1's ancestor-code-identity fallback. `scripts/firebase-distribute.sh` Gate 7 still binds via literal match as before (`--head` = the commit that already carries both code and evidence). `.github/hooks/pre-push` Check 10's `--head` = the pointer-advancing commit now falls through to the ancestor fallback when the literal match misses, and — provided the pointer-advance commit's only diff from the evidence-declared ancestor is `governance/evidence` files — binds cleanly without `--no-verify`. Any production-file diff between them still refuses exactly as before.

Options 1 (gate the literal code commit) and 3 (stay strict, evidence-at-distribute-time-only forever) were rejected — see the design doc §3.1/§3.2 for the reasoning (option 1 requires a fragile fixed commit-count offset; option 3 makes `--no-verify` a routine, expected bypass, which is itself the failure mode §6.J exists to prevent).

How it is wired

- **`scripts/firebase-distribute.sh` — Phase 1 Gate 7 (§6.AK):** after the §6.P/§6.AA/§6.Z Phase-1 gates, the distribute script invokes this

gate for the version+channel being distributed (--evidence-dir="\$CHANGELOG_DIR", --strict) and **refuses the distribute** on a non-zero exit (1 uncovered claim · 2 evidence/map missing/stale/wrong-SHA · 3 internal). It gates both the client and the api-app via the app-resolved changelog dir, with no bypass flag. See docs/scripts/firebase-distribute.sh.md for the Gate 7 detail.

- **.github/hooks/pre-push — Check 10 (per spec §5.4):** the spec defines a pre-push Check 10 that runs this gate (--strict) when a pushed commit advances a .lava-ci-evidence/distribute-changelog/<channel>/last-version pointer, so a version-advance (signalling "ready to distribute") cannot be pushed without the covering device evidence.

Constitutional binding (§6.AK clause 1)

No artifact may be distributed unless EVERY CHANGELOG-claimed user-visible fix for the current version has an EXECUTED + PASSED covering device Challenge in the matching §6.Z evidence file for the SAME commit SHA.

The CHANGELOG's user-visible bullets are the claim-set; the gate's executed-PASS Challenge set MUST be a superset of the claim-set's covering Challenges. A Challenge00-only (cold-start-only) evidence file NEVER satisfies this gate when the cycle claims flow/UI fixes — that is exactly the 1076 incident this gate exists to evict.

Maintenance

When this script is modified, update this document in the same commit (CM-SCRIPT-DOCS-SYNC / .github/hooks/pre-push Check 9 require it). Per §11.4.18 the documentation MUST stay in sync with the codebase. The gate's own hermetic test is tests/cycle-coverage/test_cycle_coverage.sh (spec §5.2 case matrix: positive + six negatives); keep its fixtures in lockstep with any change to the file formats or exit-code contract above.

Cross-references

- scripts/check-cycle-coverage.sh — the script itself
- tests/cycle-coverage/test_cycle_coverage.sh — hermetic falsifiability test

- docs/superpowers/specs/2026-06-26-ak-cycle-coverage-spec.md — the §6.AK spec (gate contract §5.1, test matrix §5.2, hook §5.4)
- docs/scripts/firebase-distribute.sh.md — Phase 1 Gate 7 wiring
- docs/helix-constitution-gates.md — gate inventory
- Lava CLAUDE.md §6.AK (Cycle-Coverage Device Gate) + §6.AK-debt
- HelixConstitution Constitution.md §11.4.18 (script documentation mandate)